Data Structure using C

Unit-1: Introduction to Data Structures

Basic Terminology – Classification of Data Structures – Abstract Datatype (ADT) – Linked Lists – Singly Linked Lists – Circular Linked Lists – Doubly Linked Lists – Applications of Linked Lists-Fundamentals of the Analysis of Algorithmic Efficiency – Asymptotic notations and their significance -complexity analysis of algorithms, worst case and average case

Basic Terminology

- **Data**: Raw facts or figures (e.g., numbers, characters) that can be processed.
- **Data Item**: A single unit of data (e.g., roll number).
- **Group Items**: Related data items grouped together (e.g., name, marks).
- **Entity**: A real-world object represented in memory (e.g., student).
- **Attribute**: A property of an entity (e.g., age, name).
- **Data Structure**: A particular way of organizing and storing data so it can be accessed and modified efficiently.
- **Operations on Data Structures**:
  - Traversing
  - Insertion
  - Deletion
  - Searching
  - Sorting
  - Merging

Types of Data Structures

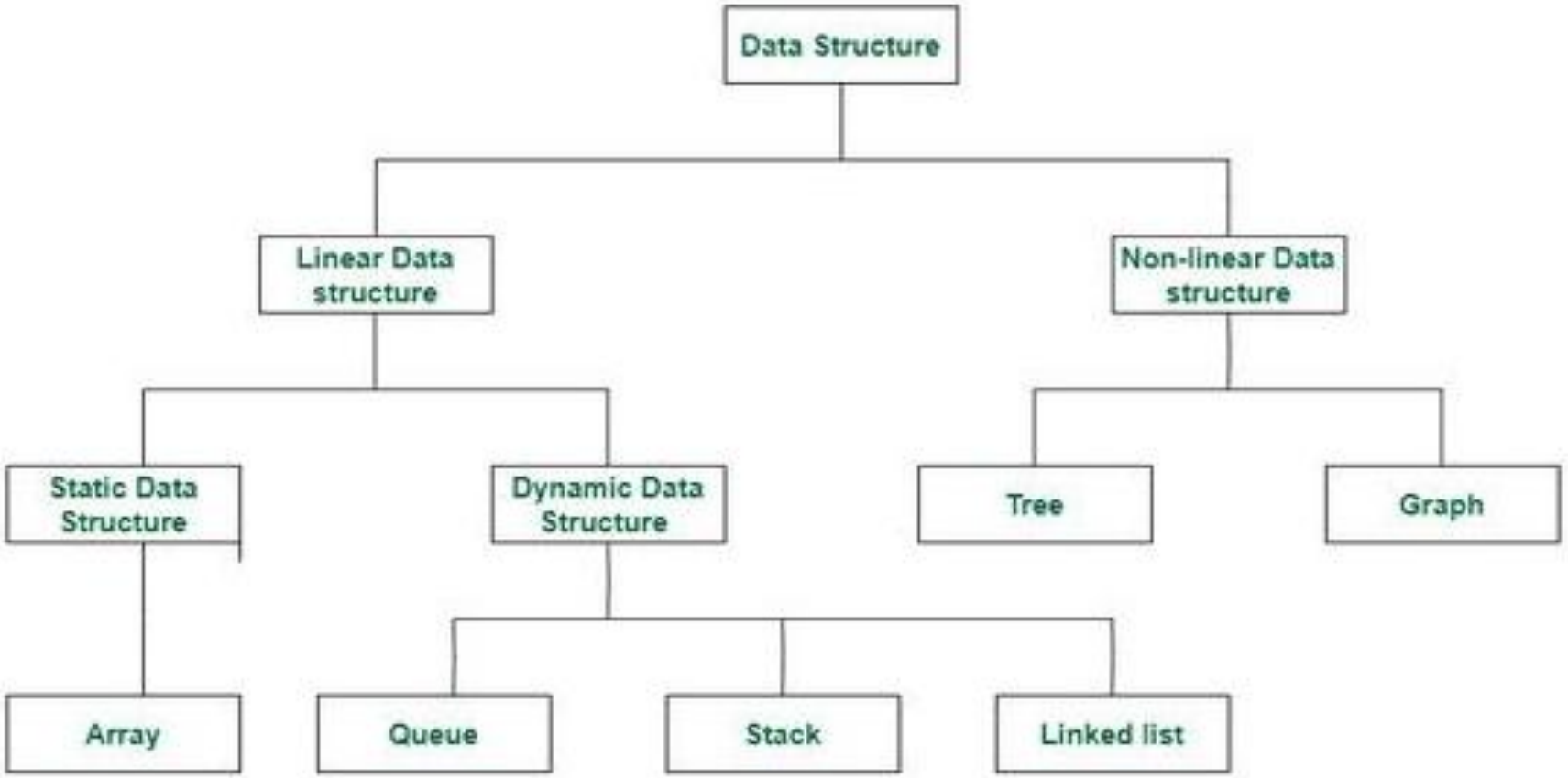
1. Primitive Data Structures

- These are the **basic data types** provided by most programming languages.

Type	Example
Integer	int, long
Float	float, double
Character	char
Boolean	true/false

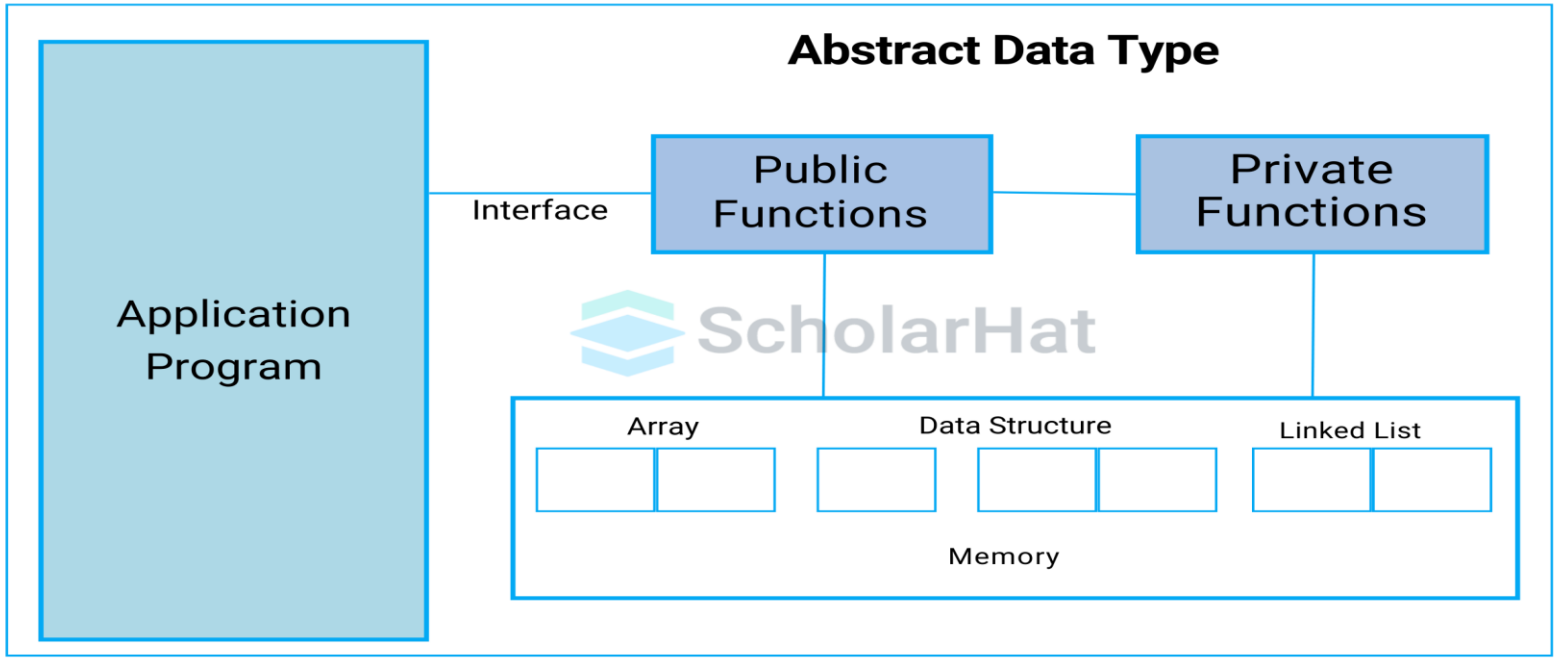
2. Non-Primitive Data Structures

These are **more complex** and are built using primitive types.



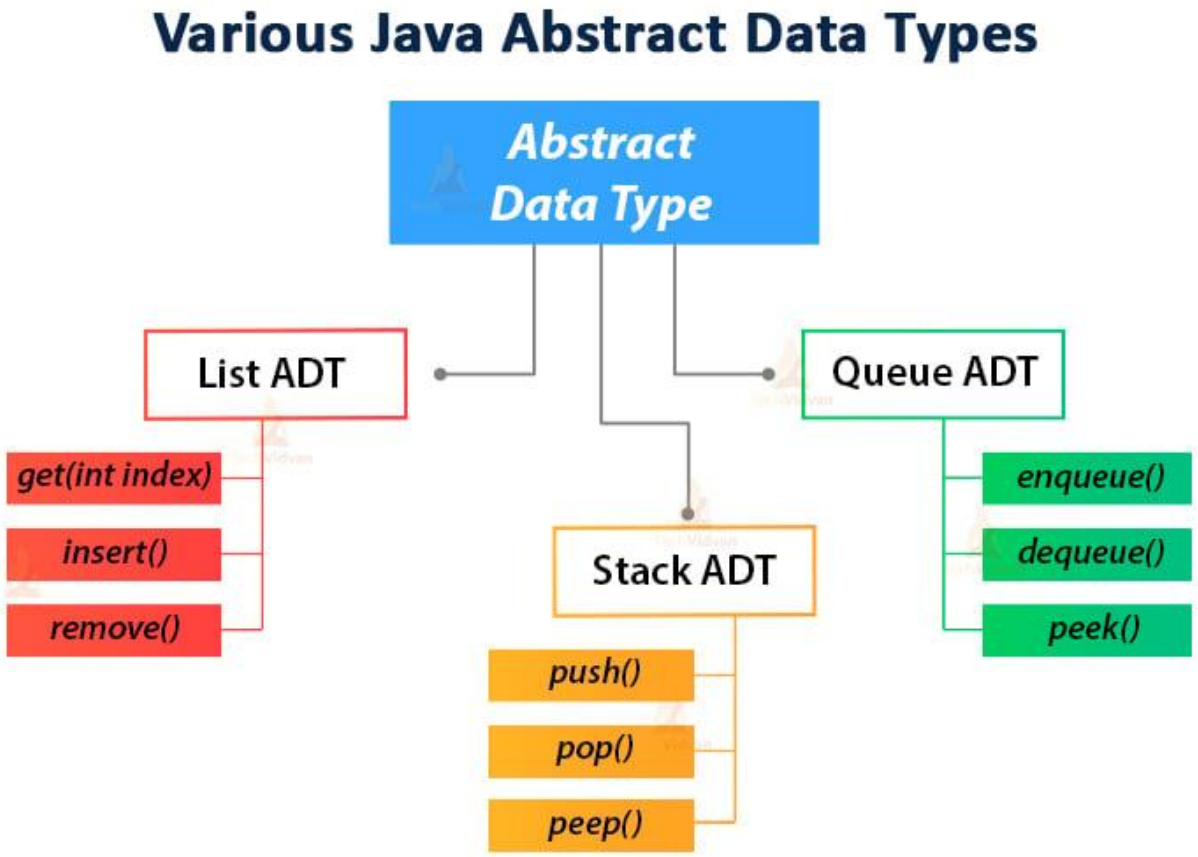
Abstract Data Type (ADT)

- An **ADT** is a logical description of a data type without specifying its implementation.
- It defines:
  - **Data** (what it holds)
  - **Operations** (what can be done with it)



❖ **Examples:**

- **List ADT:** Operations like insert, delete, search.
- **Stack ADT: LIFO** (Last In First Out) structure. Operations: push(), pop(), peek().
- **Queue ADT: FIFO (First In First Out).** Operations: enqueue(), dequeue().
- **Tree ADT: Hierarchical data structure.** Operations: insert, traverse.



Why use ADTs?

- **Code Reusability:**

ADTs promote code reusability as they can be implemented and reused in different parts of a program or in different programs.

- **Modularity:**

ADTs enhance modularity by separating the specification of a data type from its implementation, making code easier to understand, maintain, and modify.

- **Data Hiding and Encapsulation:**

ADTs encapsulate data and operations, hiding the internal representation and implementation details from the user.

- **Flexibility:**

ADTs allow for flexibility in choosing the most appropriate implementation for a specific context without affecting the rest of the program.

- **Abstraction:**

ADTs hide the complexity of the data structure, allowing users to focus on the essential operations without getting bogged down in implementation details.

Why Linked List?

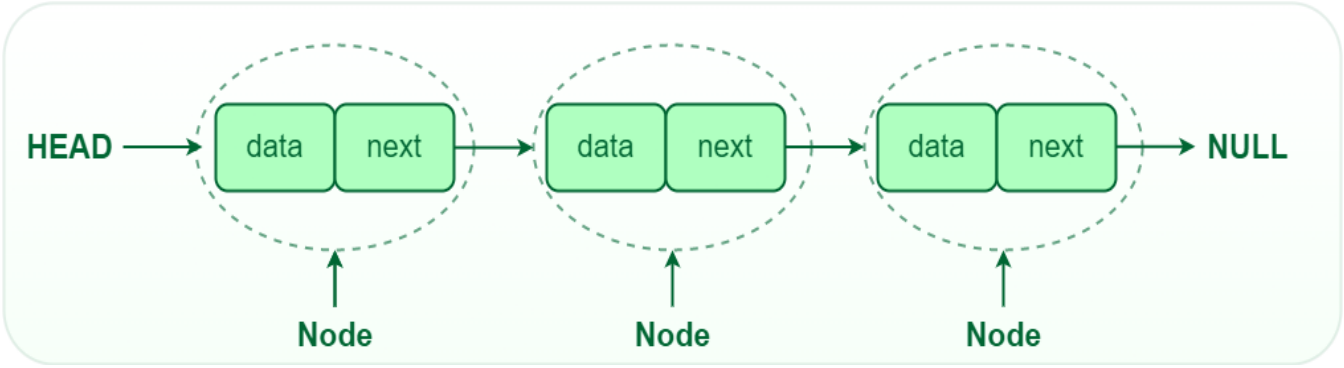
- **Arrays have the following limitations**
  - The size of the array is fixed
  - To insert a new element in an array, existing elements have to be shifted
- **Advantages of LL over arrays**
  - Dynamic size
  - Ease of insertion/deletion
  - Random access is not allowed

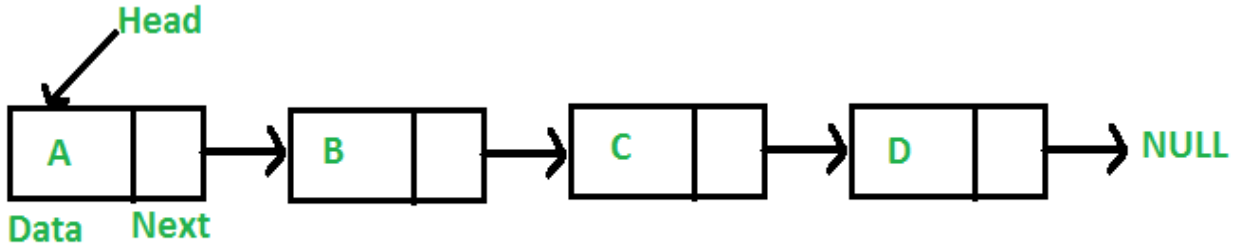
Linked List vs Array

Array	Linked List
data structure that contains a collection of similar type of data elements	non-primitive data structure contains a collection of unordered linked elements known as nodes
Accessing an element in an array is fast	Accessing an element in an array is bit slower.
Operations in arrays consume a lot of time	operations in Linked lists is fast
Arrays are of fixed size.	Linked lists are dynamic and flexible and can expand and contract its size.
In array, memory is assigned during compile time	Linked list it is allocated during execution or runtime.
Elements are stored consecutively in arrays	Elements are stored randomly in Linked lists.
requirement of memory is less due to actual data being stored within the index in the array	more memory in Linked Lists due to storage of additional next and previous referencing elements.
memory utilization is inefficient in the array	memory utilization is efficient in the linked list.

What is a Linked List?

- A linked list is a sequence of nodes in which each node is made up of two parts:
- **Data**: The value stored in the node.
- **Pointer**: A reference to the next node in the sequence. *(There can be multiple pointers for different kind of linked list.)*



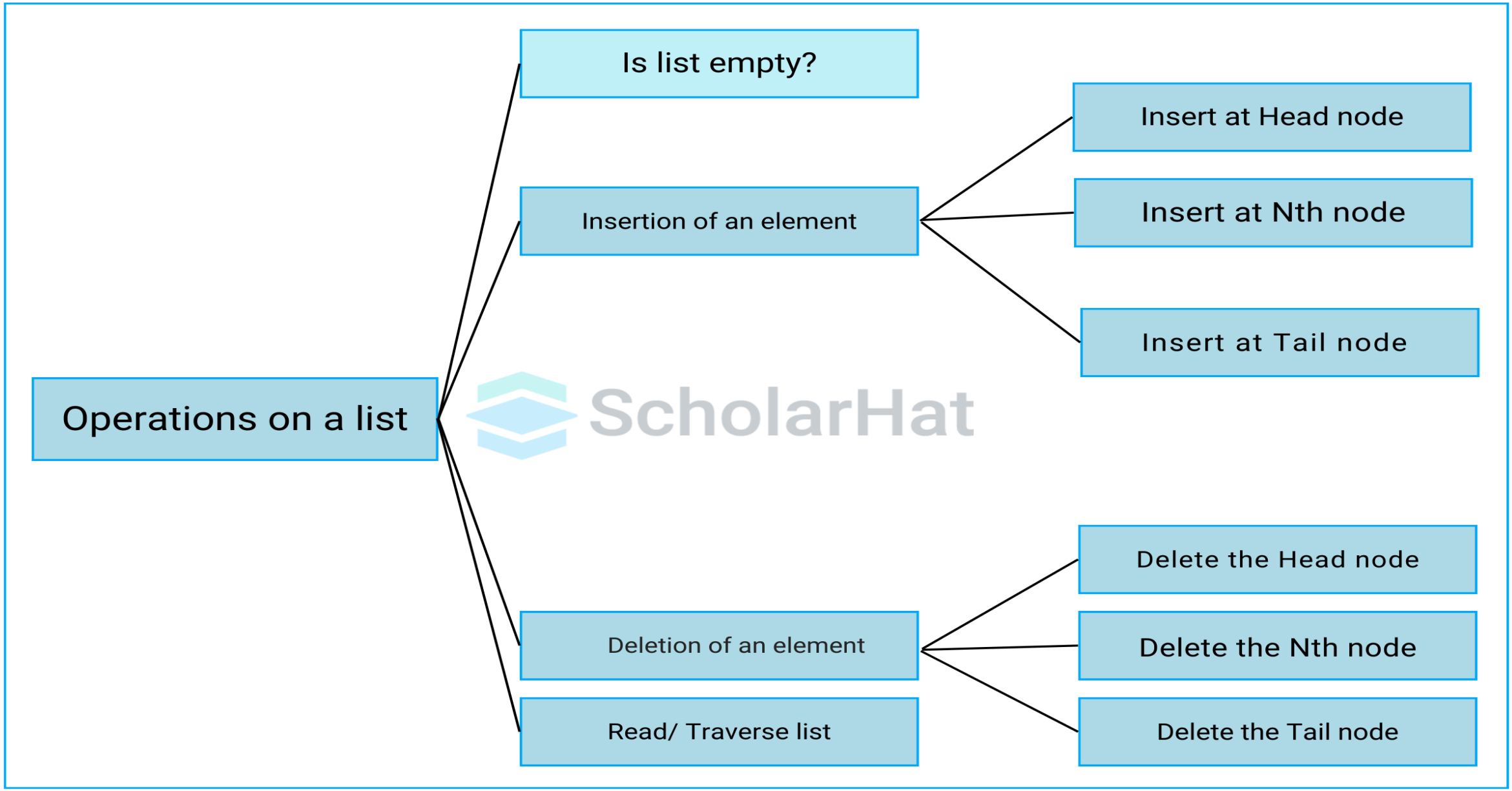


Operations on Linked Lists

1. Insertion: Adding a new node to a linked list involves adjusting the pointers of the existing nodes to maintain the proper sequence. Insertion can be performed at the beginning, end, or any position within the list

2. Deletion: Removing a node from a linked list requires adjusting the pointers of the neighboring nodes to bridge the gap left by the deleted node. Deletion can be performed at the beginning, end, or any position within the list.

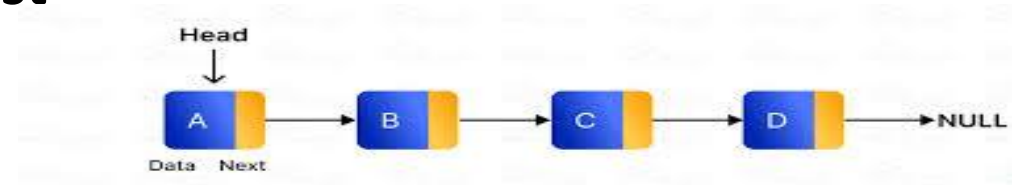
3. Searching: Searching for a specific value in a linked list involves traversing the list from the head node until the value is found or the end of the list is reached.


Operations on a list
Is list empty?
Insertion of an element
Insert at Head node
Insert at Nth node
Insert at Tail node
Deletion of an element
Delete the Head node
Delete the Nth node
Delete the Tail node
Read/ Traverse list
ScholarHat

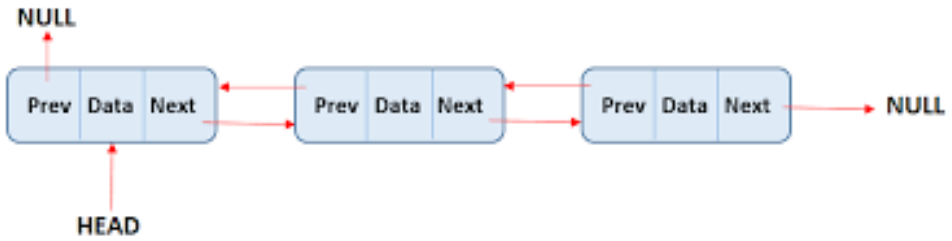
Types of Linked List in C

Linked list can be classified on the basis of the type of structure they form as a whole and the direction of access. Based on this classification, there are three types of linked lists:

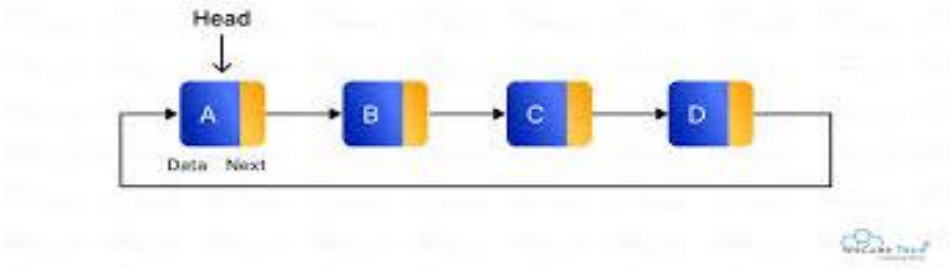
1. **Singly Linked List**



2. **Doubly Linked List**



3. **Circular Linked List**



SINGLY LINKED LIST

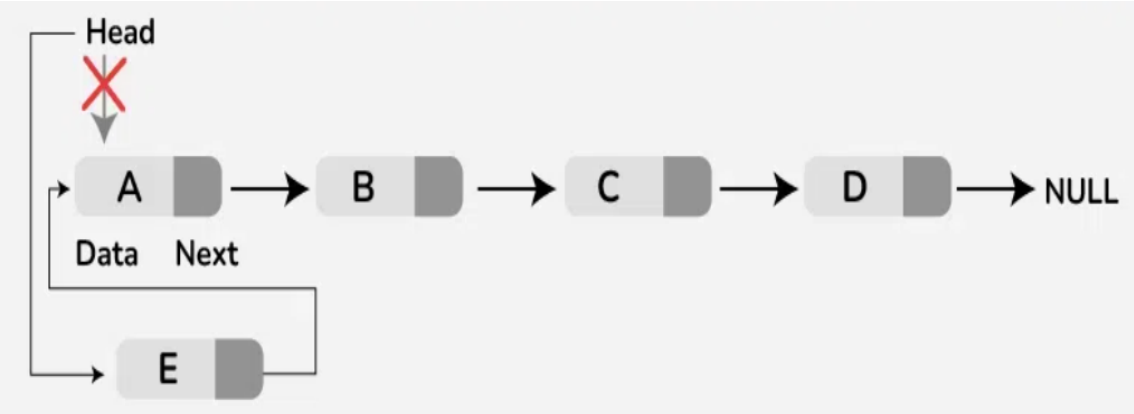
1. Singly Linked List

In a singly linked list, each node points to the **next** node. The last node points to **NULL**.

```
struct Node {
    int data;
    struct Node* next;
};
```

[10 | *] → [20 | *] → [30 | *] → NULL

Insertion at the Beginning of Singly Linked List:



Step-by-step approach:

Create a new node with the given value.

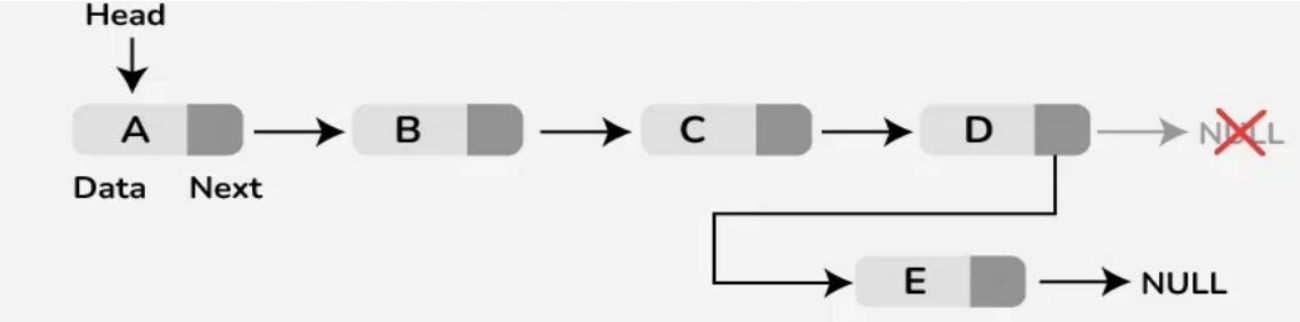
Set the **next** pointer of the new node to the current head.

Move the head to point to the new node.

Return the new head of the linked list.

```
//  Insertion at the beginning
void insertAtBeginning(struct Node** head, int data)
{
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = *head;
    *head = newNode;
}
```

Insertion at the End of Singly Linked List:



Step-by-step approach:

Create a new node with the given value.

Check if the list is empty:

If it is, make the new node the head and return.

Traverse the list until the last node is reached.

Link the new node to the current last node by setting the last node's next pointer to the new node.

```
// Insertion at the end
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

Insertion at a Specific Position of the Singly Linked List:



Step-by-step approach:

- **Create a new node** and assign it a value.
- **If inserting at the beginning** (position = 1):
  - Point the new node's next to the current head.
  - Update the head to the new node.
  - **Return** (Insertion done).
- **Otherwise, traverse the list**:
  - Start from the head and move to the (position - 1)th node (just before the desired position).
  - If the position is beyond the list length, **return an error or append at the end**.
- **Insert the new node**:
  - Point the new node's next to the next node of the current position.
  - Update the previous node's next to the new node.
- **Return the updated list.**

```
// Insertion at a given position (1-based index)
void insertAtPosition(struct Node** head, int data, int position) {
    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }

    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;

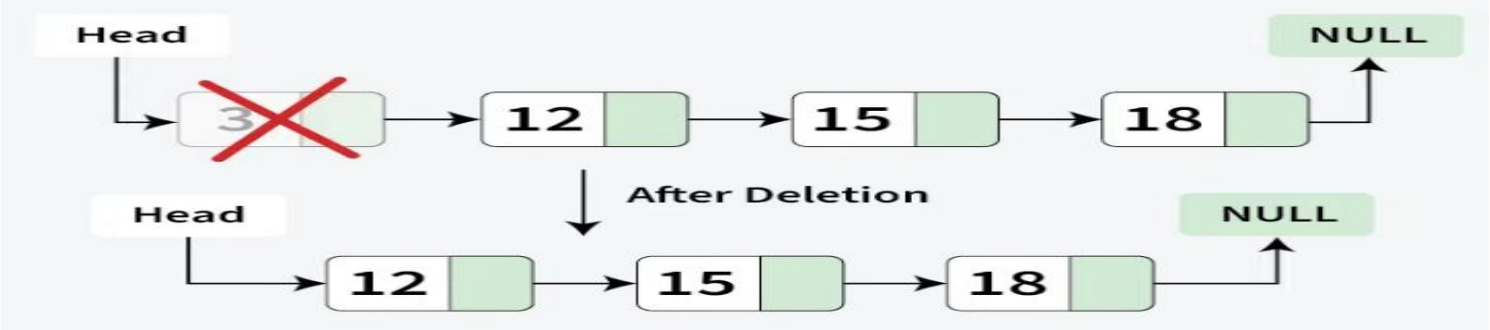
    if (position == 1) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

 if (temp == NULL) {
        printf("Position out of bounds!\n");
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
}
```

Deletion in Singly Linked List

Deletion at the Beginning of Singly Linked List:



Steps-by-step approach:

- Check if the head is **NULL**.

  If it is, return **NULL** (the list is empty).
- Store the current head node in a temporary variable **temp**.
- Move the head pointer to the next node.
- Delete the temporary node.
- Return the new head of the linked list.

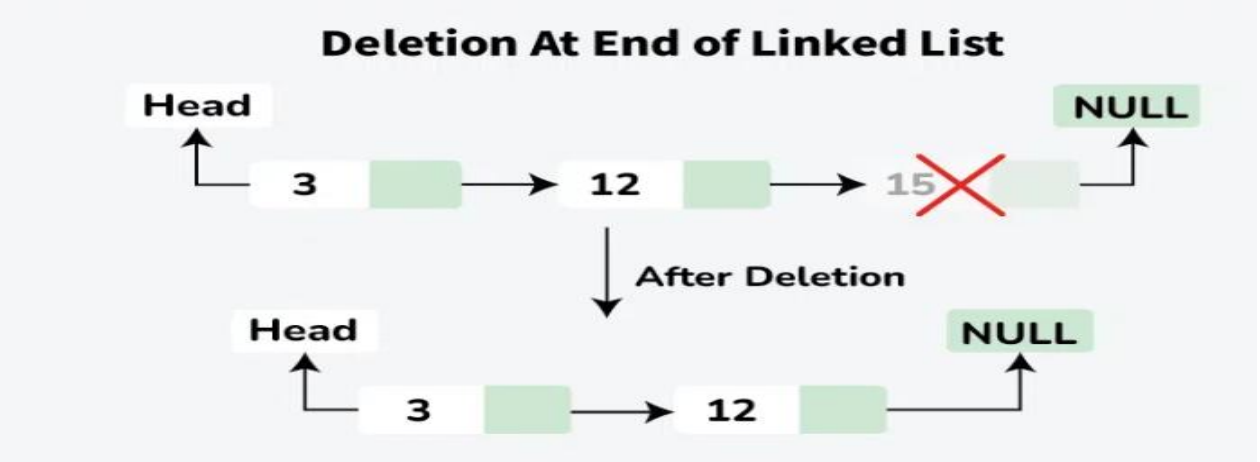
// Deletion from the beginning

```
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}
```

Deletion at the End of Singly Linked List:

To delete the last node, traverse the list until the second-to-last node and update its next field to None.



Step-by-step approach:

- Check if the head is **NULL**.
  - If it is, return NULL (the list is empty).
- Check if the head's **next** is **NULL** (only one node in the list).
  - If true, delete the head and return **NULL**.
- Traverse the list to find the second last node (**second_last**).
- Delete the last node (the node after **second_last**).
- Set the **next** pointer of the second last node to **NULL**.
- Return the head of the linked list.

Delete from End

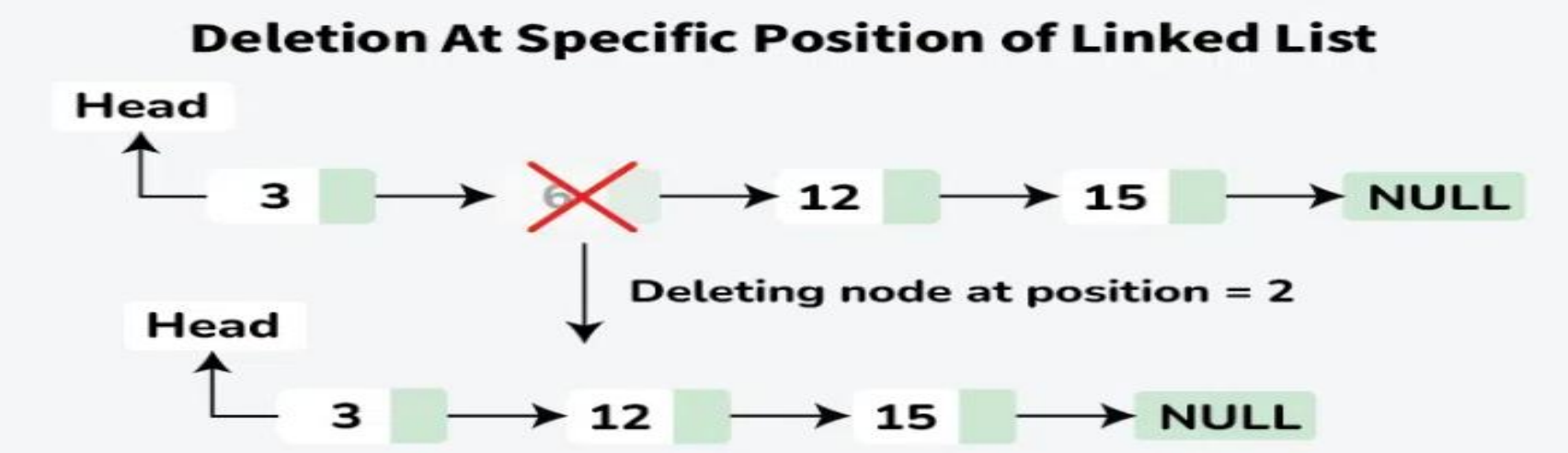
```
//  Deletion from the end
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty!\n");
        return;
    }

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
}
```

Deletion at a Specific Position of Singly Linked List:

To delete a node at a specific position, traverse the list to the desired position, update the links to bypass the node to be deleted.



- **Step-by-step approach:**
- Check if the list is empty or the position is invalid, return if so.
- If the head needs to be deleted, update the head and delete the node.
- Traverse to the node before the position to be deleted.
- If the position is out of range, return.
- Store the node to be deleted.
- Update the links to bypass the node.
- Delete the stored node.

```
// Deletion by key (first occurrence)
void deleteAtPosition(struct Node** head, int position) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *head;
    if (position == 0) {
        deleteFromFirst(head);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}
```

Traversal of Singly Linked List

Traversal in a linked list means visiting each node and performing operations like printing or processing data.

Step-by-step approach:

1. **Initialize a pointer** (current) to the head of the list.
2. **Loop through the list** using a while loop until current becomes NULL.
3. **Process each node** (e.g., print its data).
4. **Move to the next node** by updating current = current->next.

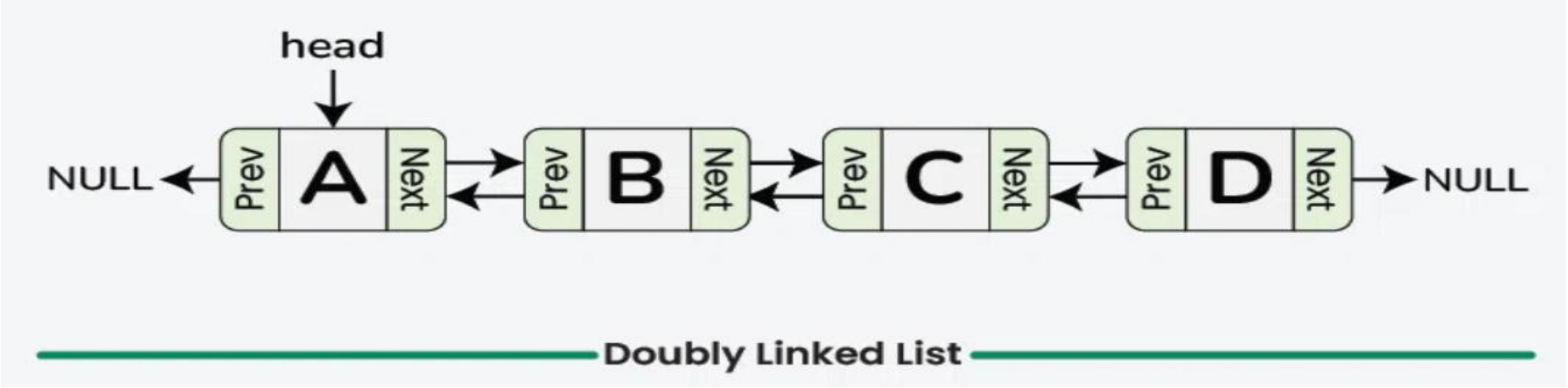
```
// Traversal: From head to NULL

void traverse(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d → ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

DOUBLY LINKED LIST

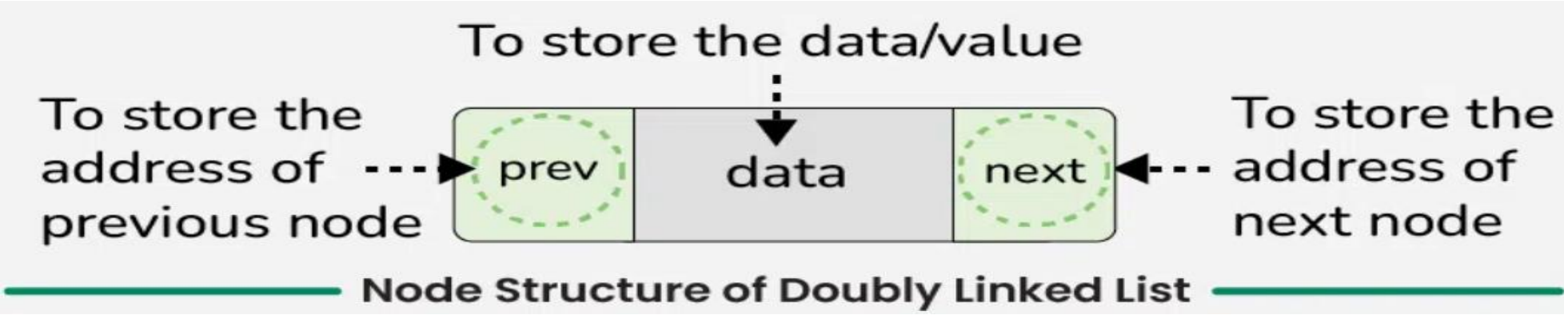
Double Linked List

- A **Doubly Linked List (DLL)** is a linear data structure in which each node has **two pointers**:
- One pointing to the **next** node
- One pointing to the **previous** node
- This allows traversal in **both directions**.



Structure of Linked List

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```

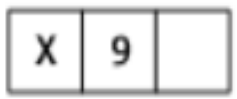


Insert at the Beginning



START

Allocate memory for the new node and initialize its DATA part to 9 and PREV field to NULL.



Add the new node before the START node. Now the new node becomes the first node of the list.



START

Insert at the Beginning

Insertion in a Doubly Linked List means adding a new node at a specific position (beginning, end, or in between). Each node maintains pointers to both its previous and next nodes, making insertion more flexible than singly linked lists.

```
void insertAtBegin(struct Node** head, int data) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = *head;

    if (*head != NULL)
        (*head)->prev = newNode;

    *head = newNode;
}
```

Insertion at the Beginning

1.Create a new node with the given data.
2.Set the next pointer of the new node to the current head.
3.If the list is not empty, update the prev pointer of the current head to point to the new node.
4.Update the head of the list to the new node.

Insert at the End



START

Allocate memory for the new node and initialize its DATA part to 9 and its NEXT field to NULL.

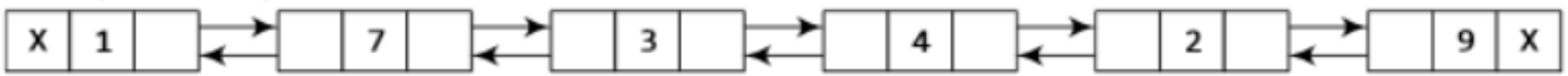


Take a pointer variable PTR and make it point to the first node of the list.



START,PTR

Move PTR so that it points to the last node of the list. Add the new node after the node pointed by PTR.



START PTR

Insert at the End

```
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        newNode->prev = NULL;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}
```

Insertion at the End

1.Create a new node with the given data.
2.If the list is empty, set the new node as the head.
3.Traverse the list until the last node is found.
4.Set the next pointer of the last node to the new node.
5.Set the prev pointer of the new node to the last node.

Insert at the Specific Position



START

Allocate memory for the new node and initialize its DATA part to 9.



Take a pointer variable PTR and make it point to the first node of the list.



START,PTR

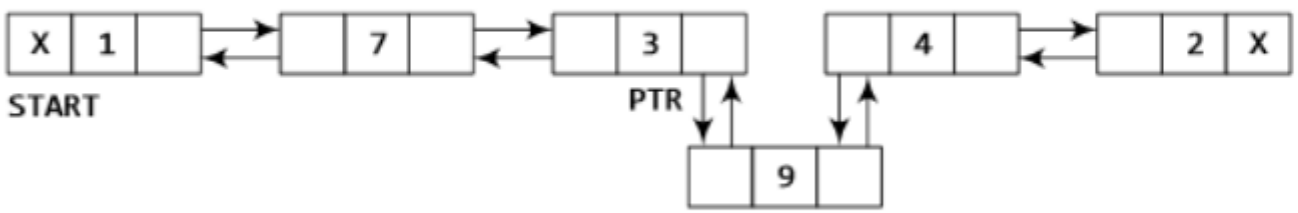
Move PTR further until the data part of PTR = value after which the node has to be inserted.



START

PTR

Insert the new node between PTR and the node succeeding it.





START

Insert at Specific Position

```
void insertAtPosition(struct Node** head, int data, int position) {
    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;

    // Insert at beginning
    if (position == 1) {
        newNode->prev = NULL;
        newNode->next = *head;
        if (*head != NULL)
            (*head)->prev = newNode;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 1; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;
}
```

Delete from Beginning

Deletion in a Doubly Linked List means removing a node while maintaining the proper linkage between the prev and next pointers of adjacent nodes.

```
void deleteFromBegin(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = temp->next;
    if (*head != NULL)
        (*head)->prev = NULL;
    free(temp);
}
```

Deletion at the Beginning

1.Check if the list is empty; if it is, return as there is nothing to delete.
2.Store the current head node in a temporary variable.
3.Move the head pointer to the next node.
4.If the new head exists, update its prev pointer to NULL.
5.Delete the old head node to free memory.



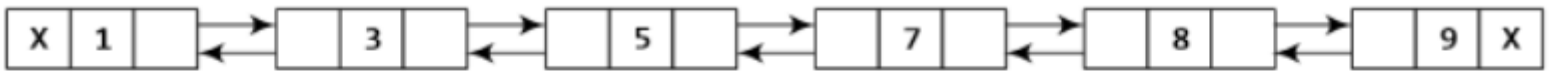
START

Free the memory occupied by the first node of the list and make the second node of the list as the START node.



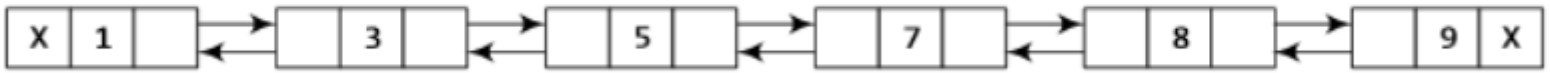
START

Delete from the End



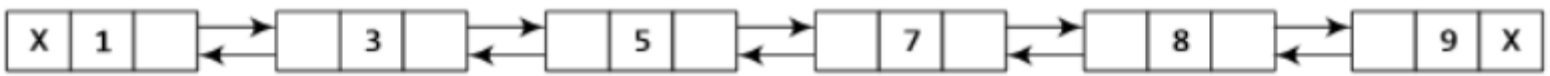
START

Take a pointer variable PTR that points to the first node of the list.



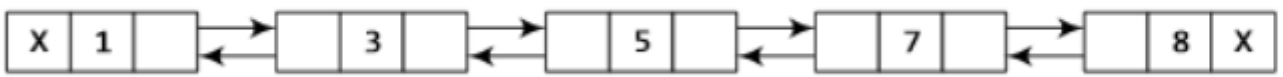
START,PTR

Move PTR so that it now points to the last node of the list.



START PTR

Free the space occupied by the node pointed by PTR and store NULL in NEXT field of its preceding node.



START

Deleting the last node from a doubly linked list

Delete from the End

```
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) return;

    struct Node* temp = *head;

    // If only one node
    if (temp->next == NULL) {
        *head = NULL;
        free(temp);
        return;
    }

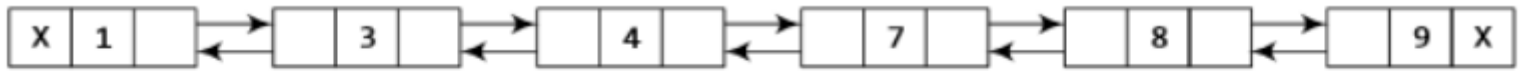
    // Traverse to the last node
    while (temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;
    free(temp);
}
```

Deletion at the End

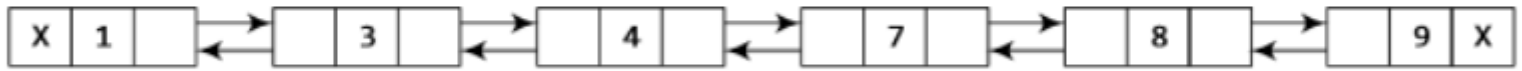
1. Check if the list is empty; if it is, return.
2. Traverse the list to find the last node.
3. Store the last node in a temporary variable.
4. Update the next pointer of the second-last node to NULL, making it the new tail.
5. Delete the last node to free memory.

Delete from the Specific Position



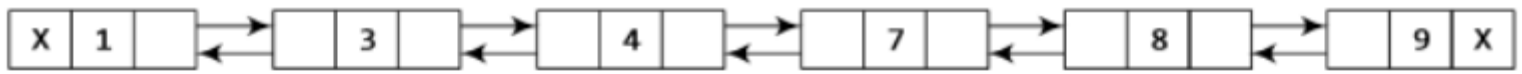
START

Take a pointer variable PTR that points to the first node of the list.

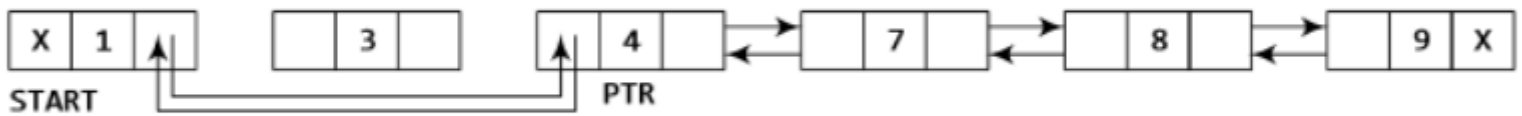


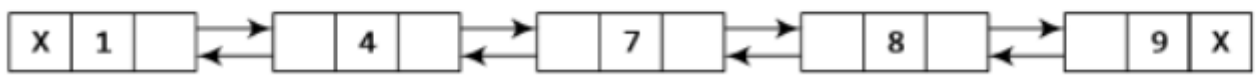
START,PTR

Move PTR further till its data part is equal to the value before which the node has to be deleted.



Delete the node preceding PTR.





START

Delete at Specific Position

```
void deleteAtPosition(struct Node** head, int position)
{
    if (*head == NULL || position < 1) {
        printf("Invalid position!\n");
        return;
    }

    struct Node* temp = *head;

    // Deleting the first node
    if (position == 1) {
        *head = temp->next;
        if (*head != NULL)
            (*head)->prev = NULL;
        free(temp);
        return;
    }

    // Traverse to the desired position
    for (int i = 1; temp != NULL && i < position; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range!\n");
        return;
    }

    // Adjust pointers
    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    free(temp);
}
```

Traversal Function

1. Forward Traversal

```
void traverseForward(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Step-by-Step Approach for Traversal:

1. **Start from the head** of the list.
2. **Traverse forward**:
   - Visit the current node and process its data (e.g., print it).
   - Move to the next node using current = current->next.
   - Repeat the process until the end of the list (current == NULL).

2. Reverse Traversal

```
void traverseReverse(struct Node* head) {
    if (head == NULL) return;

    struct Node* temp = head;

    // Go to last node
    while (temp->next != NULL)
        temp = temp->next;

    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}
```

Traverse backward:

- Start from the tail (last node).
- Visit the current node and process its data.
- Move to the previous node using current = current->prev.
- Repeat the process until the beginning of the list (current == NULL).

CIRCULAR LINKED LIST

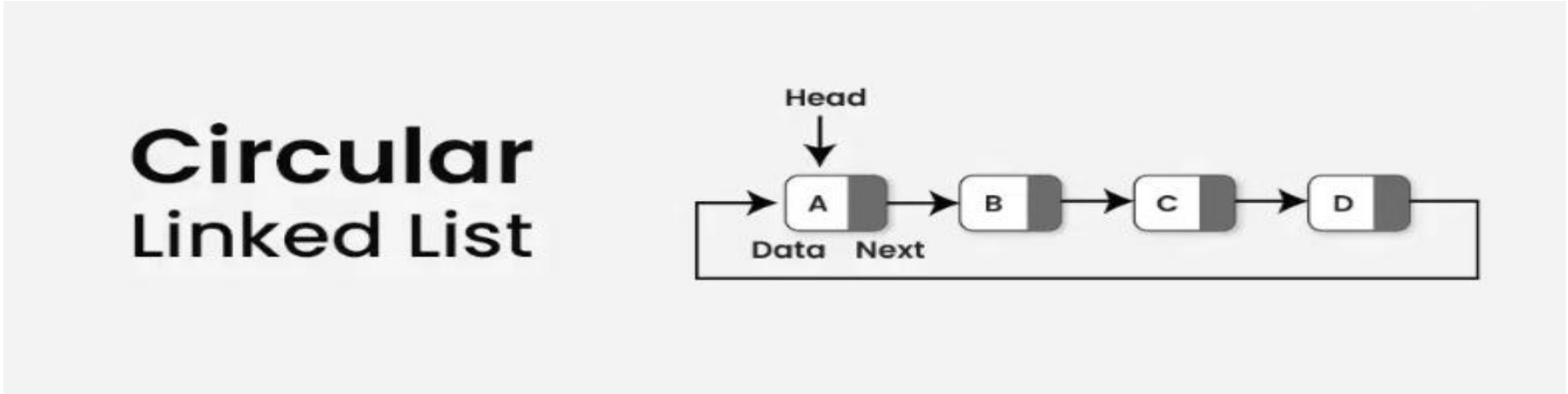
Circular Linked List

A **Circular Linked List** is a variation of a linked list where the **last node points back to the first node**, forming a circle.

It can be:

- **Singly Circular Linked List (SCLL)**: Last node's next points to the head.

- **Doubly Circular Linked List (DCLL)**: Last node's next points to head, and head's prev points to last node.



Singly Circular Linked List

Implementation of InsertAtBegin

To insert a node at the beginning of a circular linked list:

- Create a new node with the given data.
- If the list is empty, set the new node to itself and make it the head node.
- If the list is not empty, traverse to the last node and update its next pointer to point to the new node. Then update the head pointer to the new node.'

```
// Function to insert a node at the beginning
void insertAtBeginning(struct Node** head, int data)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
    }
    else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
        *head = newNode;
    }
}
```

Implementation of InsertAtEnd

To insert a node at the end of a circular linked list:

- Create a new node with the given data.
- If the list is empty, set the new node to itself and make it the head node.
- If the list is not empty, traverse to the last node.
- Set the last node's next pointer to the new node.
- Set the new node's next pointer to the head to maintain circularity.

```
// Function to insert a node at the end
void insertAtEnd(struct Node** head, int data)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
    }
    else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
    }
}
```

Implementation of InsertAtSpecificPosition

To insert a node at a specific position in a circular linked list:

- Create a new node with the given data.
- If the list is empty and position is 0, set the new node to itself and make it the head node.
- Traverse to the node just before the desired position.
- Insert the new node by adjusting pointers.

```
// Function to insert a node at a specific position
void insertAtPosition(struct Node** head, int data,
                      int position)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL && position == 0) {
        *head = newNode;
        newNode->next = *head;
    }
    else if (position == 0) {
        insertAtBeginning(head, data);
    }
    else {
        struct Node* temp = *head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;
            i++;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}
```

Implementation of DeleteFromBeginning

To delete a node from the beginning of a circular linked list:

- If the list is empty, return.
- If the list has only one node, delete it and set head to NULL.
- Traverse to the last node and update its next pointer.
- Free the memory allocated to the node to be deleted.

```
// Function to delete a node from the beginning
void deleteFromBeginning(struct Node** head)
{
    if (*head == NULL) {
        return;
    }
    else if ((*head)->next == *head) {
        free(*head);
        *head = NULL;
    }
    else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = (*head)->next;
        struct Node* toDelete = *head;
        *head = (*head)->next;
        free(toDelete);
    }
}
```

Implementation of DeleteFromEnd

To delete a node from the end of a circular linked list:

- If the list is empty, return.
- If the list has only one node, delete it and set head to NULL.
- Traverse to the second last node and update its next pointer to NULL.
- Free the memory allocated to the last node.

```
// Function to delete a node from the end
void deleteFromEnd(struct Node** head)
{
    if (*head == NULL) {
        return;
    }
    else if ((*head)->next == *head) {
        free(*head);
        *head = NULL;
    }
    else {
        struct Node* secondLast = *head;
        while (secondLast->next->next != *head) {
            secondLast = secondLast->next;
        }
        struct Node* last = secondLast->next;
        secondLast->next = *head;
        free(last);
    }
}
```

Implementation of DeleteFromSpecificPosition

To delete a node from a specific position in a circular linked list:

- If the list is empty, return.
- Traverse to the node just before the desired position.
- Adjust pointers to skip the node to be deleted.
- Free the memory allocated to the node to be deleted.

```
// Function to delete a node from a specific position
void deleteAtPosition(struct Node** head, int position)
{
    if (*head == NULL) {
        return;
    }
    else if (position == 0) {
        deleteFromBeginning(head);
    }
    else {
        struct Node* temp = *head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;
            i++;
        }
        struct Node* toDelete = temp->next;
        temp->next = temp->next->next;
        free(toDelete);
    }
}
```

Traversal

Traversing the circular linked list means visiting the each node starting from any node and continuing until all the nodes have been
visited at the least once. this operation can typically starts at the head and continuous until the head is encountered again.

Implementation of Traversal

1.Check if list is empty then if it is return.
2.Initialize the temporary pointer to the head.
3.Loop through the list starting from head and print the each nodes data.
4.Continue until the pointer reaches back to head.

```
// Function to traverse and print the circular linked list
void traverse(struct Node* head)
{
    if (head == NULL) {
        return;
    }
    struct Node* temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("HEAD\n");
}
```

Time & Space Complexity Comparison Table:

Operation	SLL	DLL	SCLL
Insert at Head	O(1)	O(1)	O(1)
Insert at Tail	O(n)	O(n) or O(1)*	O(n)
Insert at Pos	O(n)	O(n)	O(n)
Delete at Head	O(1)	O(1)	O(1)
Delete at Tail	O(n)	O(1)	O(n)
Delete at Pos	O(n)	O(n)	O(n)
Search	O(n)	O(n)	O(n)
Traversal	O(n)	O(n)	O(n)
Space per node	O(1) (1 ptr)	O(1) (2 ptrs)	O(1) (1 ptr)

Advantages and Disadvantages

Type	Advantages	Disadvantages
SLL	Simple, low memory usage	No backward traversal, tail insertion O(n)
DLL	Bi-directional, faster deletion at tail	More memory usage, complex code
SCLL	Circular structure, constant cycling	Hard to manage without stopping condition
DCLL	Full flexibility, fast operations both ends	Most memory used, slightly complex logic

Applications of Linked Lists

1. Dynamic Memory Allocation

- **Why:** Unlike arrays, linked lists don't need fixed size.
- **Example:** Managing memory in operating systems.
- **Type Used:** Singly Linked List (SLL)

2. Implementation of Data Structures

- Many abstract data types use linked lists:
  - **Stacks:** Using SLL or DLL (for push/pop).
  - **Queues:** Using SLL (for enqueue/dequeue).
  - **Graphs:** Adjacency list uses SLL/DLL for representing edges.
  - **Hash Tables:** Chaining uses SLL/DLL for collision resolution.

3. Manipulation of Polynomials

- Polynomials like $3x^2 + 2x + 1$ are stored as nodes with (coefficient, exponent).
- Enables addition and multiplication by traversing linked terms.
- **Type Used:** SLL

4. Music or Image Viewer Applications

- Moving **forward/backward** between media files (like Next/Previous).
- **Type Used:** Doubly Linked List (DLL)

5. Browser History Navigation

- Go **back and forward** through visited pages.
- **Type Used:** DLL

6. Undo/Redo Functionality

- Text editors (e.g., Microsoft Word) use DLL to track changes.
- Move backward for undo and forward for redo.
- **Type Used:** DLL

7. Scheduling Algorithms

- Operating system **task scheduling** uses Circular Linked List (CLL).
- Tasks are executed in round-robin fashion.
- **Type Used:** Circular Linked List

8. Real-Time Simulations / Gaming

- Continuous rotations (like in animations or multiplayer games).
- Circular links allow infinite looping.
- **Type Used:** Circular Linked List

9. Implementation of Sparse Matrices

- Only non-zero elements are stored in a list.
- Saves memory for large matrices.
- **Type Used:** SLL or DLL

Application of Linked List

Application Area	Linked List Type	Purpose
Dynamic memory allocation	SLL	Flexible size memory blocks
Stacks & Queues	SLL/DLL	Push/pop or enqueue/dequeue operations
Polynomial Arithmetic	SLL	Storing variable-length expressions
Music/Video players	DLL	Next and Previous controls
Browser history	DLL	Navigation between pages
OS Task Scheduling	Circular Linked List	Round-robin CPU scheduling
Undo/Redo in apps	DLL	Backward and forward operations
Sparse Matrices	SLL/DLL	Efficient storage of data

Fundamentals of Algorithm Analysis

- **What is Algorithmic Efficiency?**

Algorithmic efficiency measures how effectively an algorithm uses:

1. **Time** (execution speed)
2. **Space** (memory usage)

We analyze algorithms **independent of machine** or programming language by using **mathematical notations**.

Asymptotic Notations

Asymptotic notation describes the behavior of an algorithm as **input size n becomes very large**. It helps classify algorithms by their **growth rates**.

Types of Asymptotic Notation:

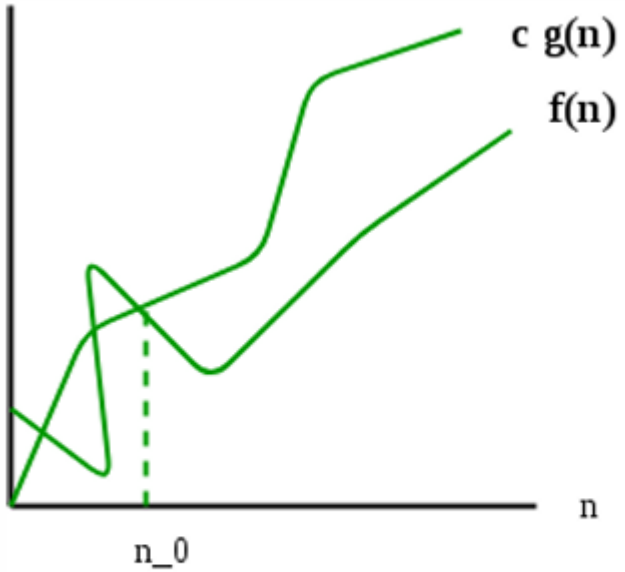
Notation	Meaning	Example
Big-O (O)	Upper bound (worst case)	$O(n^2)$
Omega (Ω)	Lower bound (best case)	$\Omega(n)$
Theta (Θ)	Tight bound (average case)	$\Theta(n \log n)$

Big O Notation (O)

- Describes the **worst-case** time complexity.
- Example:
  In linear search:

```
for (int i = 0; i < n; i++) {
    if (a[i] == key) return i;
}
```

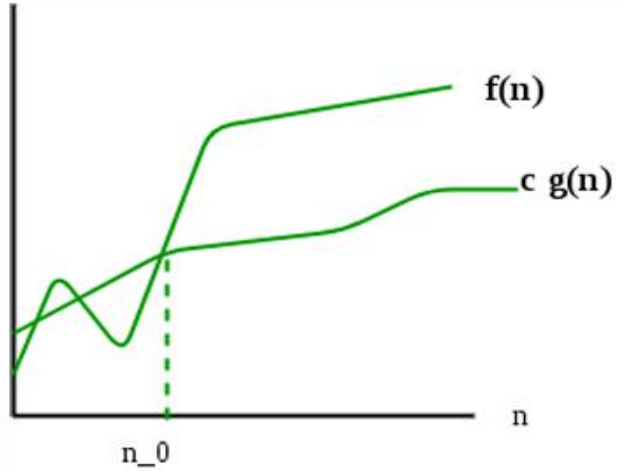
Time complexity: **O(n)**



f(n) = O(g(n))

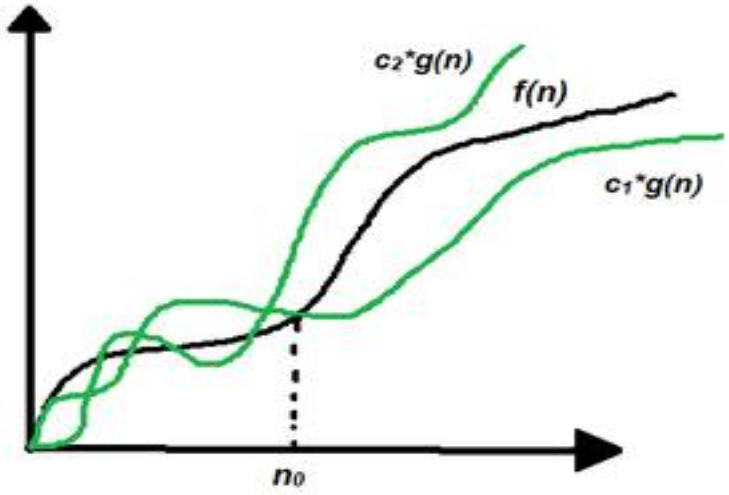
Omega Notation (Ω)

- Describes the **best-case** time complexity.
- In the same linear search, if the element is at the first position:
  Ω(1)



Theta Notation (Θ)

- Describes the **average-case** or exact bound.
- Linear search average case:
  Θ(n/2) = Θ(n)



Time Complexity Analysis of Algorithms

Time Complexity

Refers to how the **running time** of an algorithm grows with input size n.

Constant Time – O(1):

```
int a = 5;
```

Linear Time – O(n):

```
for (int i = 0; i < n; i++) {}
```

Quadratic Time – $O(n^2)$:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) {}
```

- **Logarithmic – O(log n)** – Binary Search
- **Linear Logarithmic – O(n log n)** – Merge Sort
- **Exponential – $O(2^n)$** – Recursive Fibonacci

Space Complexity

- Refers to how much **memory** is used.
- Example:

```
int arr[n];   // O(n) space
int x = 5;    // O(1) space
```

Case	Meaning	Example
Best	Fastest case (minimum operations)	Key at 1st index → Ω(1)
Worst	Slowest case (maximum operations)	Key at last or not found → O(n)
Average	Expected performance across all inputs	Avg. comparisons → Θ(n/2)

Example: Linear Search on Array of Size n

```
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}
```

- **Best Case:** Key at position 0 → **Ω(1)**
- **Worst Case:** Key not present or at end → **O(n)**
- **Average Case:** Key somewhere in middle → **Θ(n)**

Significance of Asymptotic Analysis

- **Machine-independent comparison**
- Focus on **input growth**
- Predicts scalability
- Useful in choosing the most efficient algorithm

Case	**Description**	**Example**
Best Case	Minimum steps	$\Omega(n)$
Worst Case	Maximum steps	$O(n^2)$
Average	Expected number of steps	$\Theta(n \log n)$